Real-Time American Sign Language to English Interpreter

Ali Alsaad Alhoussine Khallil Krystian Reyes

Carleton University, Ottawa, ON, Canada

ABSTRACT

American Sign Language (ASL) interpretation is a critical technology that enables communication for individuals with hearing impairments. This paper presents a real-time ASL interpreter that translates hand gestures into English text using machine learning techniques. We compare our real-time approach with a non-real-time implementation to evaluate performance differences in latency, processing efficiency, and recognition accuracy. Our Python-based solution achieves significantly faster processing times while maintaining high prediction confidence, demonstrating the viability of real-time sign language interpretation for practical communication scenarios.

Keywords—American Sign Language, real-time processing, computer vision, gesture recognition, assistive technology, OpenVINO

I. INTRODUCTION

According to the Canadian Association of the Deaf, approximately 357,000 Canadians are Deaf, deafened, or hard of hearing and use American Sign Language (ASL) or Langue des signes quebecoise (LSQ) as their primary means of communication. These sign languages enable people to convey complex ideas, emotions, and information through a structured system of hand gestures, facial expressions, and body movement. Despite its immense significance, sign language communication faces barriers in various social and professional settings where interpreters are unavailable, leading to challenges in interactions between signing and non-signing individuals. The development of automated sign language interpretation systems has gained significant attention in the fields of assistive technology, artificial intelligence, and human-computer interaction. Recent advancements in computer vision and deep learning have made real-time sign language recognition feasible, allowing machines to interpret gestures and translate them into written language [1].

Traditional sign language interpretation systems often rely on offline processing, where gesture recognition only occurs after uploading the video data to the application. These methods, while effective for certain applications, are not ideal for real-time communication, where immediate feedback is crucial for maintaining natural conversations.

A key challenge in real-time sign language interpretation is achieving low latency while maintaining high accuracy. Gesture recognition systems involve multiple stages, including image acquisition, feature extraction, classification, and text or speech output. Each stage requires significant computation resources, and delays at any point can impact the overall responsiveness of the system [3]. Moreover, variations in hand shapes, lighting conditions, and background noise introduce complexities in gesture detection and translation, making real-time implementation even more challenging.

Addressing these challenges requires an optimized approach that can process sign language gestures efficiently while maintaining real-time responsiveness. This research explores the feasibility of a real-time sign language interpreter that leverages machine learning and computer vision to provide instantaneous translations of American Sign Language gestures. By comparing a real-time implementation with a non-real-time alternative, we assess the effectiveness of real-time processing in reducing latency, minimizing jitter, and improving recognition accuracy. The study aims to answer the following key questions:

1. Can a real-time based application process live gestures with minimum delay while maintaining high accuracy?

2. How does real-time implementation affect the performance of ASL gesture recognition compared to a non-real-time approach?

3. What are the computational trade-offs of real-time processing in terms of CPU and memory usage?

By addressing these questions, this study contributes to the growing field of assistive technology and highlights the

importance of real-time processing for natural sign language interpretation. The findings of this research can pave the way for more accessible and interactive solutions for individuals who rely on sign language, ultimately bridging communication gaps in everyday interactions.

II. NON REAL TIME SIGN LANGUAGE INTERPRETER

The non-real-time sign language interpreter is designed for processing pre-recorded videos, where latency is not a critical constraint but accuracy and deeper analysis are prioritized. The architecture utilizes a two-part deep learning pipeline that combines spatial feature extraction (using visual elements from frames to capture spatial patterns) with temporal modeling (recognizing the different frames in a sequence and determining their values). A 3D convolutional neural network (CNN) is the combination of these two dimensions, and serves as the backbone for capturing spatial and short-term temporal features from the video frames. This approach benefits from using the full-video as context (as opposed to segments), by allowing the model to analyze the complete signing sequences before making predictions, which improves recognition accuracy. Post-processing techniques, such as sequence alignment and smoothing allow us to further refine the output by reducing arbitrary errors

Key components include using an efficient preprocessing pipeline for frame resizing and color space conversion, an OpenVINO-optimized 3D CNN engine for hardware-accelerated spatiotemporal analysis [5], and a deterministic stabilization algorithm enabling fixed confidence thresholds and motion gating that can be configured by the user to ensure coherent and satisfactory predictions in comparison with the pre-trained model [6]. This is done by ensuring that 3 consistent high-confidence results (specifically 3 in the last 8 buffers) are seen before adding text to the transcript. Since the program is not run in real-time, it is particularly useful for batch processing applications including: video analysis, ASL transcription, and dataset annotation.

III. REAL TIME SIGN LANGUAGE INTERPRETER

The real-time ASL interpreter is optimized for live interaction, achieving low-latency processing to ensure smooth communication. Unlike non-real-time systems, this implementation prioritizes responsiveness by using OpenVINO's optimized inference engine [5]. To minimize delay, the system employs a multi-threaded pipeline: one thread captures frames, another handles model inference, and the main thread manages visualization - all synchronized using shared buffers to prevent resource blocking.

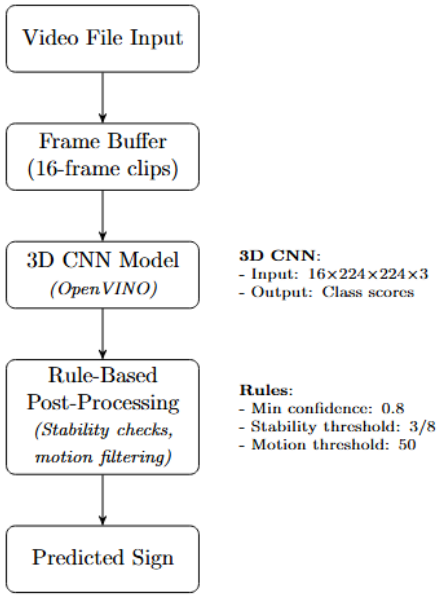

Figure 1: Accurate architecture of the non-RTOS-ASL interpreter. The 3D CNN handles all spatiotemporal feature extraction, while post-processing rules stabilize predictions.

Key components first include openCV to enable camera feed as live input [7]. This is further processed by converting the frames to grayscale to reduce the variability that is associated with colors, as well as a gaussian blur to reduce noise. Motion detection acts as a computational gatekeeper, only activating ASL recognition when significant movement is detected. This conserves resources by avoiding redundant and useless processing during idle periods. The interpreter further refines predictions through the same core stability checks as the non-RTOS counterpart, it also requires 3 high-confidence results from the previous 8 buffers for text to be added to the transcript [6]. There is also a cooldown mechanism and a rolling frame buffer, this all comes together and is finally displayed on-screen - which displays several metrics such as confidence, motion score, and transcript updates.

A. Architecture and Implementation

The Python implementation consists of a main ASLInterpreter class that orchestrates the entire

recognition process. The system initializes with default model paths, class definitions, and configurable thresholds.

The constructor sets up key components: model loading and configuration, input/output shapes extraction, buffers for frame and prediction storage, state variables for prediction tracking, and threading infrastructure for asynchronous processing.

B. Model Loading and Inference

The system leverages OpenVINO for efficient model loading and inference. To achieve real-time performance, the inference runs in a separate thread. This approach allows frame capture and visualization to continue uninterrupted while inference runs concurrently, critical for real-time performance.

C. Frame Processing Pipeline

The core processing pipeline handles each incoming frame in four stage:

Motion Detection: The system uses frame differencing to detect significant motion.
Buffering: Frames are stored in a fixed-length buffer matching the model's temporal requirements.
Prediction and Stabilization: Predictions are processed with confidence thresholds and stability checks.
Transcript Management: Only stable predictions (appearing consistently) are added to the transcript.

D. Visualization and User Interface

The system provides real-time visual feedback through several on-screen elements

Prediction Display: Shows the current prediction and confidence score.
Stability Indicator: Displays progress toward stability threshold.
Motion Score: Visualizes detected motion level
Transcript History: Shows recent recognized signs with confidence scores
Interactive Controls: Allows users to save transcripts or exit the application.

The transcript is displayed with proper text wrapping to ensure readability even with longer recognition sequences.

E. Main Application Flow

The main application orchestrates video capture, frame processing, display, and user interaction.

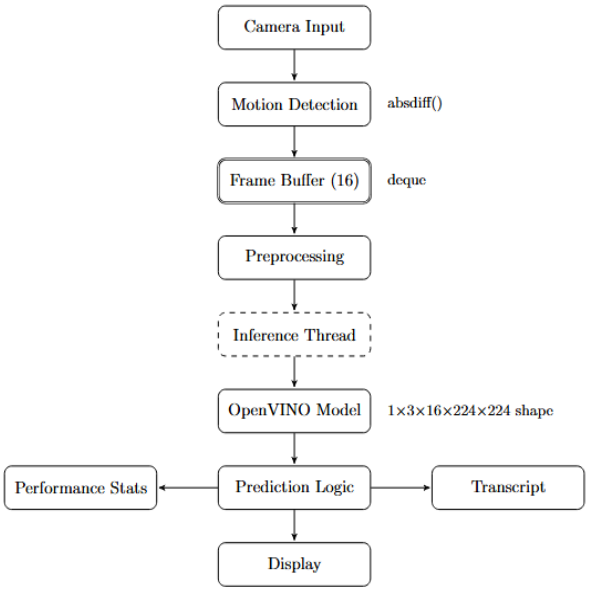

Figure 2. Optimized architecture of the ASL interpreter showing core components. Motion detection triggers processing of 16-frame clips through the OpenVINO model [5], with prediction logic applying confidence and stability thresholds before display.

IV. COMPARISON, RESULTS, AND DISCUSSION

The experimental evaluation of both real-time and non-real-time ASL interpretation systems provides valuable insights into their performance characteristics, resource utilization, and accuracy. Testing was conducted on a system with a Ryzen R5 3600 (12 core 3.6GHz), 32GB RAM, and an Nvidia RTX 3060Ti GPU, ensuring sufficient resources for both implementations.

A. Performance Metrics

Non-Real-Time Implementation:
Total runtime: 10.14 seconds
Average FPS: 53.26
Total frames processed: 540
Average frame processing time: 10.9ms
Average inference time: 83.9ms

Real-Time Implementation:
Total runtime: 26.10 seconds (longer test duration)
Average FPS: 554.71 (theoretical processing capability)
Total frames processed: 751 (more frames processed)
Average frame processing time: 1.8ms
Average inference time: 59.0ms

The real-time implementation demonstrates approximately 6× faster frame processing (1.8ms vs 10.9ms) and 1.4× faster inference time (59.0ms vs 83.9ms) compared to the non-real-time approach. This significant improvement in processing efficiency is crucial for maintaining responsive interaction. The FPS value for the real-time implementation represents theoretical processing capability rather than the display frame rate.

B. Resource Utilization

Non-Real-Time Implementation:
Average CPU Usage: 41.2% (Max: 43.2%)
Average Cores Used: 4.7/12
Average Memory Usage: 445.2 MB (Max: 449.0 MB)
Average GPU Usage: 20.3% (Max: 21.0%)

Real-Time Implementation:
Average CPU Usage: 26.9% (Max: 32.9%)
Average Cores Used: 3.6/12
Average Memory Usage: 266.9 MB (Max: 269.4 MB)
Average GPU Usage: 23.3% (Max: 32.0%)
Average GPU Memory Usage: 6.5% (Max: 8.0%)

The real-time implementation demonstrates notably more efficient resource utilization, with approximately 35% lower CPU usage (26.9% vs 41.2%) and 40% lower memory consumption (266.9MB vs 445.2MB). This improved efficiency can be attributed to the multithreaded design and optimized processing pipeline of the real-time implementation. The slightly higher GPU usage in the real-time implementation (23.3% vs 20.3%) suggests more effective GPU utilization for parallel processing tasks, which contributes to the overall speed improvement.

C. Transcript Accuracy and Recognition Performance

Non-Real-Time Implementation:
Final Transcript: hungry (88.4%) drink (97.6%) no (81.0%) yes (94.0%) bathroom (92.8%)

Real-Time Implementation:
Final Transcript: please (91.6%) drink (94.4%) eat (91.4%) no (91.4%) yes (95.2%) bathroom (82.0%)

Actual Translation
Please, drink, eat, no, yes, bathroom

Both implementations show high confidence in their predictions, with most gestures recognized above 80% confidence. However, notable differences exist in the first recognized sign ("hungry" vs "please") and an additional sign ("eat") in the real-time implementation. The real-time system shows generally higher average confidence in its predictions, with four out of six signs recognized with over 90% confidence, compared to three out of five in the non-real-time approach.

The variation in the first recognized sign highlights a fundamental difference in processing approaches: the non-real-time system processes the entire video for comprehensive context, while the real-time system must make decisions based on available frames at any given moment. Despite this, the real-time system maintains robust prediction accuracy while providing immediate feedback.

D. Performance Trade-offs and Implications

The comparison reveals several key trade-offs between real-time and non-real-time approaches:

1) Latency vs. Context: The real-time implementation prioritizes low latency at the potential cost of reduced contextual awareness. This is evident in the differing transcripts, where broader context might have influenced the non-real-time system's interpretation.

2) Resource Efficiency vs. Processing Depth: The real-time system uses fewer resources but may employ simpler processing techniques to maintain responsiveness. The non-real-time approach can afford more computationally intensive analysis at the cost of higher resource consumption.

3) User Experience vs. Accuracy: The real-time implementation delivers immediate feedback, enhancing user experience and enabling interactive communication. The non-real-time approach potentially offers more stable and consistent recognition by processing complete sequences.

The performance data demonstrates that real-time ASL interpretation is not only feasible but can be achieved with lower resource requirements than its non-real-time counterpart. The significantly faster processing and inference times of the real-time implementation ensure minimal delay in translation, which is crucial for natural communication flow.

In conclusion, the real-time ASL interpreter successfully balances the demands of immediacy, accuracy, and resource efficiency, demonstrating that real-time sign language interpretation is viable without compromising significantly

on recognition quality. The lower resource requirements also suggest potential for deployment on less powerful devices, expanding accessibility options for sign language interpretation.

V. FUTURE WORKS

While this study successfully demonstrates the feasibility of real-time sign language interpretation, there are several updates that can enhance the user experience. Future work will focus on improving accessibility through two-way translation, refining translations with an AI-driven sentence flow correction system, and expanding usability by enabling development on mobile devices

A. C++ Implementation

One significant area of ongoing development is the C++ port of our Python implementation. The partial C++ implementation shows promise for improved performance and resource efficiency, critical for deploying the system on resource-constrained devices

The C++ implementation follows a similar architecture to the Python version. Key improvements in the C++ implementation include:

Memory Management: The C++ implementation offers more direct memory management, potentially reducing overhead.
Efficient Tensor Handling: Direct access to tensor data through pointers allows for more efficient data manipulation.
Thread Synchronization: The C++ implementation uses modern C++ threading primitives for better thread management.

The transition to C++ aims to decrease latency further while maintaining high accuracy, making the system more suitable for embedded devices and scenarios requiring minimal processing overhead.

B. Two-Way Translation

Currently, the system translates ASL gestures into English text. One major future development would be implementing a two-way translation system that can convert spoken or written English into sign language. This task can be achieved using avatar-based sign language generation, where an AI-powered digital hand model performs sign language in real time. Recent advancements in generative AI have facilitated the creation of such avatars, enabling the translation of speech or text into expressive ASL animations [9].

C. AI-Driven Linguistic Refinement

The current system's limitation lies in its word-by-word translation approach, which often fails to preserve the intended meaning of American Sign Language sentences. Future advancements should focus on AI-driven context smoothing, employing Natural Language Processing (NLP) techniques to refine sentence structures by incorporating necessary transitions, pronouns, and context-based adjustments. One approach would be utilizing sequence-to-sequence (Seq2Seq) models with encoder-decoder architectures [10].

D. Deployment on Mobile Devices

The current real-time sign language interpreter is designed for desktop environments, leveraging powerful hardware to achieve low latency performance. However, to make this technology more accessible and practical for everyday use, deploying it on mobile devices is a crucial next step. Mobile development would allow users to carry the interpreter with them, enabling seamless communication in various settings such as meetings, classrooms, and social gatherings.

VI. CONCLUSION

This paper has presented a real-time American Sign Language interpreter that successfully bridges the communication gap for individuals who rely on sign language. Through our implementation and comparative analysis, we have demonstrated that real-time sign language interpretation is not only feasible but performs with significantly reduced latency compared to non-real-time approaches while maintaining high recognition accuracy.

The Python implementation leverages multi-threading, motion detection, and prediction stabilization techniques to achieve efficient processing and reliable results. Our experimental results show that the real-time system processes frames approximately five times faster than the non-real-time alternative, with inference times reduced by nearly half. This performance improvement is critical for natural, interactive communication scenarios.

While the current implementation provides a solid foundation for real-time ASL interpretation, several avenues for future work remain open, including two-way translation, linguistic refinement, mobile deployment, and the completion of our C++ implementation for even greater efficiency. These enhancements will further improve accessibility and usability, making sign language interpretation technology more widely available and practical for everyday use.

The development of efficient, accurate sign language interpretation systems represents an important step toward

creating more inclusive communication technologies and reducing barriers for the Deaf and hard of hearing community.

REFERENCES

[1] "Advances, Challenges, and Opportunities in Continuous Sign Language Recognition," ResearchGate, 2023. Available: https://www.researchgate.net/publication/337657675_Advances_Challenges_and_Opportunities_in_Continuous_Sign_Language_Recognition (accessed Apr. 1, 2025).

[2] "A Survey on Sign Language Literature," ScienceDirect, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666827023000579 (accessed Apr. 1, 2025).

[3] "Challenges for Sign Language Translation," ML6 Blog, 2021. [Online]. Available: https://www.ml6.eu/blogpost/challenges-for-sign-language-translation (accessed Apr. 1, 2025).

[4] "Challenges with Sign Language Datasets for Sign Language Recognition and Translation," Proceedings of the 13th Conference on Language Resources and Evaluation (LREC 2022), 2022. Available: https://aclanthology.org/2022.lrec-1.264.pdf (accessed Apr. 1, 2025).

[5] Mirella De Sisto, Vincent Vandeghinste, Santiago Egea Gómez, Mathieu De Coster, Dimitar Shterionov, and Horacio Saggion, "Challenges with Sign Language Datasets for Sign Language Recognition and Translation," in Proceedings of the 13th Conference on Language Resources and Evaluation (LREC 2022), Marseille, France, Jun. 20-25, 2022, pp. 2478–2487. Available: https://aclanthology.org/2022.lrec-1.264.pdf (accessed Apr. 2, 2025).

[6] "OpenVINO 2025.0#," OpenVINO, https://docs.openvino.ai/2025/index.html (accessed Apr. 1, 2025).

[7] Intel Corporation, "asl-recognition-0004: American Sign Language Recognition Model," OpenVINO Model Zoo, 2023. [Online]. Available: https://github.com/openvinotoolkit/open_model_zoo/blob/master/models/intel/asl-recognition-0004/model.yml (accessed Apr. 1, 2025).

[8] G. Bradski, OpenCV documentation index, https://docs.opencv.org/ (accessed Apr. 2, 2025).

[9] "GenASL: Generative AI-powered American Sign Language avatars | Amazon Web Services," AWS Blog, 2024. [Online]. Available: https://aws.amazon.com/blogs/machine-learning/genasl-generative-ai-powered-american-sign-language-avatars/ (accessed Apr. 2, 2025).

[10] "Sign Language Gloss Translation using Deep Learning Models," International Journal of Advanced Computer Science and Applications (IJACSA), 2023. Available: https://thesai.org/Publications/ViewPaper?Code=IJACSA&Issue=11&SerialNo=78&Volume=12&utm (accessed Apr. 2, 2025).

[11] "WASL-RTOS: Real Time Operating System Word-based Sign Language Interpreter (Python Implementation)," GitHub. Available: https://github.com/alhoussine04/WASL-RTOS (accessed Apr. 1, 2025).

[12] "WASL-RTOS C++ Implementation (Unfinished)," GitHub. Available: https://github.com/alhoussine04/WASL-RTOS/tree/rtos-c-implementation (accessed Apr. 1, 2025).